

WEEK 12 · TYPESCRIPT & MOCK SURGERY 2

THE COMPILER READS IT TOO

The application is finished. Today the compiler reads one layer of it — and then you work alone for forty minutes.

WHERE WE ARE

Week 12 of 12. Six files, one loop, live data — and one layer where a value you did not write comes in.

THE NEXT 180 MINUTES

Four cycles, you typing in every one: rename and guard · type the DOM · Claude migrates a module · one refactor in three commits. Break at 85, the surgery at 115, the assignment at 165.

WHAT THIS UNLOCKS

The project's last requirement, submitted this week — and the third verb on the exam paper.

Types describe shape, not intent.

TARGET · MINUTE 0

THIS IS WHERE YOU WILL BE AT MINUTE 180

the pantry, with its network layer in TypeScript – it looks exactly like the one you start from

reload

open

המזווה הרשימה פריט סיכום

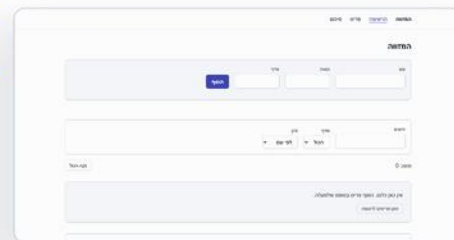
המזווה

שם	כמות	מדף	הוסף

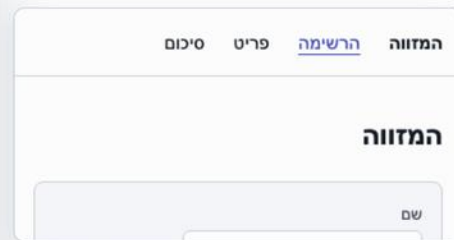
חיפוש	מדף	מיון

נקה הכול

מוצג: 0



targets/week-12.png · 1280



375 · the same shape, narrower

► **THE POINT** — the screen does not change today. What changes is that `tsc` reads `ts/api.ts`, and a 404 that is valid JSON can no longer reach the list unchecked. You type each piece once in a cycle first, on the class lending library.

AI THIS WEEK

PARTNER. IT MIGRATES; YOU READ WHAT IT SILENCED.

LAYER · WEEKS 8–12 · PARTNER

ALLOWED

Claude writes the migration — in Claude Code on the folder, or in the chat with the files pasted — **against four rules you wrote into the prompt**, and you check what came back with `grep` and `tsc` before you run it (cycle 3; cycle 2's you-do).

NOT ALLOWED

`any`, `as` on a value from outside, `@ts-ignore` — in anything you submit. **Any AI in cycles 1 and 4's "you do" and in the surgery** — alone, offline, like week 13.

NO SUBSCRIPTION?

Everything works in the free chat, and nothing in the course depends on a paid plan. Paste the files, copy back what returns, run the same review. Same grade.

CYCLE 1 · 25 MIN

RENAME, COMPILE, AND GUARD THE BOUNDARY

The class lending library, four JavaScript modules and the assignment's own `tsconfig.json`. One module — `storage.js`, where the disk comes in — moves to `ts/storage.ts`. You read the error count out loud at every run, and the screen ends exactly as it started.

1 I DO

10 MIN

3 on the next five slides, 7 in the editor · live-code/01-rename-and-compile · types.ts first · rename, run: 2, then 4 · the guard value is `ShelfEntry` · `'./types'` (wrong) · `'./types.js'`

2 WE DO

5 MIN

the same steps on your copy · say the count at every run · compare with checkpoint 1

3 YOU DO

8 MIN

the second guard: `isBook` — strict on what the list breaks without, tolerant on a missing year

4 DEBRIEF

2 MIN

zero errors and a blank page · the count that went up · ↗ reference

YOU DO · AI-FREE

➤ REFERENCE · 2-14

A TYPE IS A CLAIM. IT IS NOT CODE.

the same module, before and after tsc

reload

open

THE SAME MODULE, BEFORE AND AFTER tsc

highlight what disappeared

clear

ts/source.ts — WHAT YOU WRITE

```
// Everything in this file that describes a type is gone after compilation.
// (The class lending library's types - cycle 1 types a longer version of this file.)
export type LookupKind = 'offline' | 'status' | 'shape';

export interface ShelfEntry {
  readonly id: string;
  title: string;
  added: string;
}

function isRecord(value: unknown): value is Record<string, unknown> {
  return typeof value === 'object' && value !== null && !Array.isArray(value);
}

export function isEntry(value: unknown): value is ShelfEntry {
  if (!isRecord(value)) return false;
  return (
    typeof value.id === 'string' &&
    typeof value.title === 'string' &&
    typeof value.added === 'string'
  );
}

export function latest(entries: readonly ShelfEntry[]): { total: number; newest: string | null } {
```

js/source.js — WHAT THE BROWSER LOADS

```
function isRecord(value) {
  return typeof value === 'object' && value !== null && !Array.isArray(value);
}
export function isEntry(value) {
  if (!isRecord(value))
    return false;
  return (typeof value.id === 'string' &&
    typeof value.title === 'string' &&
    typeof value.added === 'string');
}
export function latest(entries) {
  const total = entries.length;
  const newest = entries.reduce((best, entry) => (best === null || entry.added > best ?
    entry.added : best), null);
  return { total, newest };
}
```

TypeScript never changes the runtime behaviour of JavaScript code.

TYPES FIRST, THEN THE ONE MODULE WHERE OUTSIDE VALUES COME IN

1 Types first.

`ts/types.ts` — the entity, the unions, the guard.

2 One module, and the right one:

where a value you did not write comes in — `storage.js` in cycle 1, `api.js` in cycle 3, the pantry's `api.js` in the assignment.

Rename it and compile.

3 The compiler emits `js/storage.js` at the same path, so **nothing that imports it changes.**

4 Fix the first error only

, and run again. Read the count out loud.

5 Stop.

The other modules stay JavaScript, and get their types from the emitted `.d.ts` anyway.

READ THE COUNT OUT LOUD: 2, THEN 4, THEN 0

1 · RENAME → 2

`ts/storage.ts` is
`js/storage.js`, word for word.
Two errors, both `TS7006`: a
parameter with no type. **Nothing is
written to `js/`.**

2 · FIX THE FIRST → 4

`value: unknown`. Now
Property `'id'` does not exist on
three times. **The count went up** —
the compiler is asking what the
JavaScript assumed.

3 · THE GUARD → 0

`isRecord` first, then three
`typeof` checks, in
`isEntry(value: unknown): value is She`
Zero, and four files in `js/`.

► **FORTY ERRORS ARE NOT FORTY PROBLEMS.** Usually they are one, in forty places.

THE ONLY HONEST BRIDGE OUT OF `unknown`

```
1 export function isEntry(value: unknown): value is ShelfEntry {  
2   if (!isRecord(value)) return false;  
3   return (  
4     typeof value.id === 'string' &&  
5     typeof value.title === 'string' &&  
6     typeof value.added === 'string'  
7   );  
8 }  
9  
10 // and what it buys, in storage.ts:  
11 const entries = parsed.filter(isEntry); // ShelfEntry[], because of `is`
```

`: boolean`

“the answer is yes”

`: value is ShelfEntry`

“... and from here on you may treat it as one”

A VALUE IMPORT KEEPS ITS PATH

```
1 import { isEntry } from './types.js';           // a function: stays in js/storage.js
2 import type { ShelfEntry } from './types.js';    // a type: erased, gone from the output
```

WHAT TS/STORAGE.TS SAYS

NPX TSC -P TSCONFIG.JSON

THE BROWSER

from './types' — what the editor's auto-import writes

0 errors

blank page — 404 on [js/types](#)

from './types.ts'

TS5097 — refused

—

from './types.js'

0 errors

loads

The path in an `import` is the address the browser will ask for — and the browser loads `js/`.

CHECKPOINT 1

YOUR SCREEN SHOULD LOOK LIKE THIS — EXACTLY LIKE IT DID BEFORE

ספריית ההשאלה

המדף שלי

6 ספרים בקטלוג - שורה אחת לא נקראה

שמרתי לקריאה

המדף ריק. הוסף ספר מהקטלוג.

הקטלוג

חיפוש בקטלוג

אור בחלון

נועה ברק · 2019 · כ-6 שעות קריאה

+ למדף

גשר על הירקון

דן אלמגור · 2008 · כ-4 שעות קריאה

+ למדף

הים של סבתא

רות גל · 2021 · כ-2 שעות קריאה

+ למדף

כוכבים בצהריים

מיכל שחר · שנה לא ידועה · כ-9 שעות קריאה

+ למדף

מפה בלי צפון

זנתון לב · 2015 · כ-3 שעות קריאה

+ למדף

שבע דלתות

עמר נור · 2003 · כ-11 שעות קריאה

+ למדף

Compare — six books and “one row could not be read”, the same as `start/`. Then: `ls js/` shows `types.js` and `storage.d.ts`; `js/types.js` has two functions and no `interface`; both imports in `storage.ts` end in `.js`.

CYCLE 2 · 25 MIN

TYPE THE DOM: ONE HONEST GENERIC

`querySelector` returns `Element | null`, and both halves are in the way. One generic function makes the assertion **once, with a name, in a place you can find** — and throws with the selector in the message when the element is not there. `app.js` stays JavaScript and imports it.

1 I DO

10 MIN

3 on the next four slides, 7 in the editor · live-code/02-type-the-dom · `querySelector(...).value : 2` errors on one line · `$` with a bare `as T` (wrong) · throw with the selector · the typo, by name

2 WE DO

5 MIN

`$`, `inputText`, the import in `app.js` · compare with checkpoint 2

3 YOU DO

8 MIN

the event half: `rowOf(event)` with `instanceof Element`, wired into both click handlers

4 DEBRIEF

2 MIN

forty as `HTMLInputElement` · the helper that returns null quietly · [reference](#)

YOU DO · AI AS PARTNER

THE CASTING YOU WILL ACTUALLY NEED

four real diagnostics, the helper, and two error messages compared

↶ reload

open ↷

ELEMENT OR NULL, AND NEITHER ONE IS WHAT YOU WANTED

A FORM TO QUERY

search קמח

- first
- second

read the box, both ways

a selector that matches nothing

RAW `querySelector`

`$<T>()` — ONE ASSERTION, IN ONE PLACE

AND WHAT `tsc --strict` SAYS — THE RAW VERSION, THE HELPER, THE EVENT

```
1 // `querySelector` returns `Element | null`, and both halves are in the way.
2 function readBox() {
3   const box = document.querySelector('#query');
4   return box.value; // two errors on one line
5 }
6
7 // One generic helper, one assertion, in a place you can find again.
8 function $<T> extends Element = Element>(selector: string): T {
9   const found = document.querySelector(selector);
10  if (found === null) throw new Error(`no element matches ${selector}`);
```

► Press **A SELECTOR THAT MATCHES NOTHING**. Only one of the two messages names the typo.

ONE GENERIC, READ ALOUD

```
1 export function $<T extends Element = Element>(  
2   selector: string,  
3   scope: ParentNode = document,  
4 ): T {  
5   const found = scope.querySelector(selector);  
6   if (found === null) throw new Error(`nothing on the page matches ${selector}`);  
7   return found as T;  
8 }  
9  
10 $<HTMLInputElement>('#q').value; // compiles: T is HTMLInputElement  
11 $('#status').textContent;        // T is Element, the default
```

T

a parameter whose value is a **type**
— the caller picks it

extends Element

some kind of element — **not**
anything: `$<number>` does not
compile

= Element

the default, so `$('main')` works
with no type argument at all

THE GENERIC THAT PROMISES WHAT IT NEVER CHECKS

two signatures, one file, and what `tsc --strict` said about both

↻ reload

open ↗

TWO SIGNATURES THE COMPILER CANNOT TELL APART

the endpoint answers:

`getJSON<T>(url): Promise<T>`

`getChecked<T>(url, guard): Promise<T>`

BOTH OF THEM, THROUGH `tsc --strict`

`tsc --strict`, over both versions: 0 errors.

```
1 interface Suggestion {
2   title: string;
3 }
4
5 // THE SIGNATURE PEOPLE WRITE FIRST. It compiles, and it is a lie: there is no `T` at
6 // runtime, so nothing here checks anything. Every caller gets a fully-typed object that
7 // may be an error document.
8 async function getJson<T>(url: string): Promise<T> {
9   const res = await fetch(url);
10  return res.json();
11 }
12
13 // THE HONEST ONE. The caller supplies the type AND the proof.
14 async function getChecked<T>(url: string, guard: (v: unknown) => v is T): Promise<T> {
15   const res = await fetch(url);
16   const body: unknown = await res.json();
17   if (!guard(body)) throw new Error('the response is not what we asked for');
```

► **VOTE FIRST.** Which of the two is the wrong one?

AND THE EVENT, WHICH IS THE SAME TRICK

NEITHER OF THESE IS RIGHT

```
1 // does not compile:
2 const row = event.target.closest('li');
3
4 // compiles, and lies:
5 const row = (event.target as HTMLElement)
6   .closest('li');
```

ONE LINE, AND IT IS TRUE AT RUNTIME

```
1 if (!(event.target instanceof Element))
2   return null;
3 // narrowed from here on:
4 return event.target
5   .closest<HTMLElement>('li[data-id]');
```

instanceof is a runtime test the compiler understands. That is **value is T** wearing a different hat.

CHECKPOINT 2

YOUR SCREEN SHOULD LOOK LIKE THIS – STILL

ספריית ההשאלה

המדף שלי

6 ספרים בקטלוג · שורה אחת לא נקראה

שמרתי לקריאה

המדף ריק. הוסף ספר מהקטלוג.

הקטלוג

חיפוש בקטלוג

אור בחלון

מועה בריק · 2019 · כ-6 שעות קריאה

למדף +

גשר על הירקון

דן אלמגור · 2008 · כ-4 שעות קריאה

למדף +

הים של סבתא

רות גל · 2021 · כ-2 שעות קריאה

למדף +

נוכבים בצהריים

מיכל שמיר · שנה לא ידועה · כ-9 שעות קריאה

למדף +

מפה בלי צפון

זונתן לב · 2015 · כ-3 שעות קריאה

למדף +

שבע דלתות

עמר נצר · 2003 · כ-11 שעות קריאה

למדף +

Compare — the same screen. Then: type the first word of one title in the search — one row; `js/dom.js` has no `as` and no `<T`; `app.js` has no `document.querySelector`; `$('#nope')` in the console throws, and the message says `#nope`.

CYCLE 3 · 25 MIN · AI

MIGRATE `api.js` — AND READ WHAT IT SILENCED

Prompt 1 is what almost everyone asks: “move it to TypeScript, make it compile”. What returns compiles with zero errors — and silences the compiler in three places, one of which deletes the guard. You find all three with the browser, `grep`, and `tsc` with the silencing removed.

1 I DO

8 MIN

3 on the next two slides, 5 in the chat · ai-lab/03-migrate-api · P1 live · paste it · `tsc : 0` · reload

2 WE DO

6 MIN

browser, `grep`, then `tsc` without the `@ts-ignore : 0`, then 1, then 3 · type the missing field yourself

3 YOU DO

9 MIN

your own P2 with four checkable rules · review before you run · one review line, in the exemplar’s form

4 DEBRIEF

2 MIN

what the `@ts-ignore` was hiding · the declaration graded this week · ↗ reference

YOU DO · AI AS PARTNER

PROMPT 1, AND WHAT CAME BACK

IT COMPILES. FIND THE THREE PLACES IT SWITCHED THE COMPILER OFF.

THE PROMPT, WORD FOR WORD

```
Move js/api.js to TypeScript. The new file:
ts/api.ts.
It has to compile with npx tsc -p tsconfig.json
- the tsconfig and ts/types.ts are below.
Keep the behaviour as it is.

<js/api.js · ts/types.ts · tsconfig.json>
```

THE ANSWER, CUT TO THE LINES THAT MATTER (WHOLE: TRANSCRIPT.MD)

```
1 constructor(kind: LookupKind, messageHe: string) {
2   super(messageHe);
3   // @ts-ignore - kind is assigned at runtime
4   this.kind = kind;
5 }
6 ...
7 } catch (error: any) {
8   ...
9   const payload: any = await res.json();
10  // The response is typed now, so every entry is a Book.
11  const books = payload.books as Book[];
12  return { books, skipped: 0 };
```

► tsc : zero errors. Reload: **"7 BOOKS IN THE CATALOGUE"** — there are six — and an empty list.

WHICH LINE LET THE ROW WITHOUT A TITLE THROUGH?

WHAT `tsc` SAYS WHEN YOU STOP SILENCING IT

TS/API.TS	NPX TSC -P TSCONFIG.JSON	WHAT IT POINTS AT
as Claude returned it	0	nothing — that is what the three silencers are for, and <code>res.json()</code> is <code>Promise<any></code> in the DOM's own types
without the <code>@ts-ignore</code>	1 — TS2339	Property 'kind' does not exist on type 'LookupError': the field was never declared
... you type <code>readonly kind</code> , then both <code>any</code> become <code>unknown</code>	2 — TS18046 ×2	'payload' is of type 'unknown' — exactly the two lines that need a guard

```
1 export class LookupError extends Error {  
2   readonly kind: LookupKind;           // you type this — and the 1 is 0
```

The count the compiler prints when nobody silences it is the work list.

CYCLE 3 · ON SCREEN BEFORE ANYONE WRITES A REVIEW

A REVIEW OF MACHINE-WRITTEN CODE THAT SOMEBODY CAN CHECK

THE GOOD ONE

The claim and the conclusion are two lines. What the explanation said; what actually runs; and **the code that proved it, with what it printed.** The two functions that were right are left alone.

THE BAD ONE

“I read the code and understood the order can change.”
No run, no output — and a claim about `Set` that is itself wrong.

► **AI-EXEMPLARS/EXEMPLARS/AI-REVIEW-JS-GOOD.MD**, then **AI-REVIEW-JS-BAD.MD**, in your course materials. Assignment 12 asks for the same shape in `PROMPTS.md`: what it wrote, what you changed, **AND WHY**.

GRADED FOR THE FIRST TIME THIS WEEK

THE DECLARATION: `AI_USAGE.md`, GOOD AND BAD

THE GOOD ONE

By file. “Can I explain every line?” is **not “yes” on every row** — where it is “partly”, it says which lines, and what you will do before week 13.

THE BAD ONE

“About 40%.” “I understand all the code: yes.” Honest, and worth nothing — to the grader, and to you a week before the exam.

► **AI-EXEMPLARS/EXEMPLARS/AI-DECLARATION-GOOD.MD**, then **AI-DECLARATION-BAD.MD**. Due with the project, this week — and the exam is where it is checked.

CHECKPOINT 3

THREE QUESTIONS, THREE YESES

You ran the compiler

1 after deleting the `@ts-ignore`

, read `TS2339`, and fixed it with a declaration — not with another silencer.

Every rule in your P2 is something `grep`, `tsc` or the browser can check, and the answer passed all four

2 before you ran it

.

Your review line says

3 what you saw

— 7 where there are 6, a console error, an error code.

BREAK

10 MINUTES

Back at 95 for one short cycle — the last one before the exam, and AI-free. At 115, the surgery: no AI, no internet, alone, forty minutes.

CYCLE 4 · 20 MIN

ONE REFACTOR, THREE COMMITS

The last cycle before the surgery, and the exam's third verb. `render.js` of the library, three smells: a number with no name, a calculation written twice, a function four indents deep. One move per commit, the screen compared after every one — and one “improvement” that is not a refactor.

1 I DO

7 MIN

2 on the next two slides, 5 in the editor · live-code/04-three-commits · three lines on the board first · name the constant · extract, with `Math.round` (wrong) · `Math.ceil`

2 WE DO

5 MIN

screenshot first, then commits 1 and 2 · compare with checkpoint 4

3 YOU DO

6 MIN

commit 3: guard clauses in `shelfNote` — every exit at the top, no `else`

4 DEBRIEF

2 MIN

“and I improved it on the way” · ↗ reference

THE EXAM'S THIRD VERB

REFACTORING

A third of the exam, and the only one of the three verbs whose success is invisible.

A refactor is a change to the shape of code that does not change its behaviour.

THE CATALOGUE – SEVEN MOVES, AND YOU NEED ALL OF THEM

THE MOVE	THE SMELL THAT CALLS FOR IT
extract function	a comment explaining what the next five lines do
rename for intent	<code>data, temp, handleClick2</code>
remove duplication	the same three lines, twice, with one word different
guard clauses over nesting	the real work is four indents from the margin
split by responsibility	a function whose name needs the word “and”
name the magic value	<code>86400000, '-', 0.6</code>
simplify the conditional	<code>if (x === true) return true; else return false;</code>

CHECKPOINT 4

THE SAME SCREEN AS YOUR SCREENSHOT, AND TWO COMMITS

Compare — with the screenshot you took before the first move, not with this picture alone: every reading time the same. Then: `grep -c "40" js/render.js` is 1; `Math.ceil` appears once; `git log --oneline` shows two commits whose names say what moved.

SURGERY · 115–165

FORTY MINUTES ALONE, EIGHT TOGETHER, THEN TEN ON THE MIGRATION

117–157 · ALONE

The full surgery. One repair, one addition, one restructure, on an application you have not seen. **No AI, no internet, no neighbour.** Two versions in the room — check that your sheet and your folder agree.

157–165 · TOGETHER

The walkthrough, straight away — **and the grader running on screen,** so you see exactly how the automatic points are awarded.

165–175 · THE ASSIGNMENT

The spec, first and by hand, and its commit. The migration itself — typed today on the library — **at home, before lesson 13.**

The three sentences on that paper are word for word the three sentences on the exam.

GOOD, VERSUS MERELY DIFFERENT

#	CRITERION	WHO CAN ANSWER IT
1	behaviour is identical — every input, every output	a machine
2	the thing written twice is now written once	a machine
3	the names say what , not how or where	a person
4	the change is the size of the task, and no larger	a person
5	a future change that touched N places now touches one	a person

These five are on the exam's answer key, in this order. They are not a secret.

ASSIGNMENT · 165-175

CLASS ASSIGNMENT 12: THE SAME MIGRATION, ON THE PANTRY

1 The spec, first · now, in class

`specs/ts-migration.md`, committed before `ts/` — no AI.

2 Types, guard, network · cycles 1 and 3

`value is Item` · `unknown at res.json()` · `readonly kind` · `'./types.js'`

3 The DOM · cycle 2

`$<T extends Element = Element>`, imported by one JavaScript file.

4 The review · cycle 3

two things the tool wrote and you changed, and why, in `PROMPTS.md`.

► **TEN MINUTES NOW:** the spec, sections 1-3, and its commit. The rest at home, **DUE BEFORE LESSON 13.** `ShelfEntry` is `Item`, `LookupError` is `ApiError` — nothing is copied; everything is recognised.

THE PROJECT IS DUE AT THE END OF THIS WEEK

BEFORE YOU PUSH

`node .checks/run-checks.mjs`
from `project/`. **The same checks**
CI runs, on your machine, in under
a minute.

THE CHECKLIST

`SUBMISSION_CHECKLIST-he.pdf`
— every requirement as something
you **do and observe**, not
something you believe.

THE ONE THAT COSTS THE MOST

Committing `ts/` and forgetting
`js/`. The browser loads the
output. **The grader does too.**

► Moodle: **REPO URL + COMMIT SHA**. The SHA freezes what is graded.

THREE QUESTIONS BEFORE YOU GO

1 You wrote `interface ShelfEntry` and compiled.
What is in the output file?

2 `tsc` printed zero errors and the page is blank.
Name two ways that happened today.

3 You migrated one file out of six.
How many are now type-checked?



BEFORE NEXT WEEK

- 1 Submit the project.**
Repo URL + SHA, on Moodle, before the deadline. Run the self-check first.
- 2 Open your own project and read it once**
, end to end, as if somebody else wrote it. That is the exam.
- 3 Nothing to install.**
The lab machines will have your project preloaded and no internet. Bring nothing.

Next week you get three sentences you have now read twice. Only the application will be new — and it will be yours.



GO DEEPER

The reference deck for this week — every slide the lecture pointed at, in order.

[2 · You have already shipped this bug](#) ← spine 4

[3 · The same file, with one word added](#) ← spine 4

[4 · Most annotations you write are already known](#) ←

spine 4

[5 · The ones you already have, named](#) ← spine 4

[6 · Shapes: interface or type](#) ← spine 4

[7 · Optional, and readonly](#) ← spine 4

[8 · You have been writing unions since week 10](#) ← spine 4

[9 · Four ways to ask what a value is](#) ← spine 4

[10 · The whole build step](#) ← spine 4

[11 · noEmitOnError: two errors, and not one file written](#)

← spine 4

[12 · Why tsc accepts './types', and the browser does not](#)

← spine 4

[13 · The two places the libraries hand you any](#) ← spine 4

[14 · Where npx tsc finds the compiler](#) ← spine 4

[15 · The DOM library already speaks the same generic](#) ←

spine 11

[16 · A function's type is its contract](#) ← spine 17

[17 · any and unknown at the one boundary that matters](#)

← spine 17

[18 · as is a declaration, not a check](#) ← spine 17

[19 · as and readonly, as the compiler reports them](#) ←

spine 17

[20 · A class field is declared, or the compiler does not](#)

[know it exists](#) ← spine 17

[21 · Two words, and both of them do work](#) ← spine 24

[22 · "Identical", measured instead of promised](#) ← spine

24

[23 · Extract, and rename — the two that pay](#)

[immediately](#) ← spine 24

[24 · Split by responsibility, and simplify the conditional](#)

← spine 24

[25 · Guard clauses, because this is the one they all need](#)

← spine 24

[26 · How to refactor safely when you have no tests](#) ←

spine 24

[27 · Four submissions, graded in front of you](#) ← spine 24